In [ ]:
```
import cv2   #Pragya singahl, CU24220050, 43
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, clear_output
```

In [ ]:
```
video_path = "video_preview_h264.mp4"       #Pragya singahl, CU24220050, 43
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: Video file could not be opened.")
else:
    # Get video properties
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    print("Video loaded successfully!")
    print(f"Total Frames : {total_frames}")
    print(f"FPS          : {fps}")
    print(f"Resolution   : {width} x {height}")

cap.release()
```

```
Video loaded successfully!
Total Frames : 126
FPS          : 23.976023976023978
Resolution   : 960 x 540
```

In [ ]:
```
cap = cv2.VideoCapture(video_path)     #Pragya singahl, CU24220050, 43
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out_lk = cv2.VideoWriter(
    "lucas_kanade_output.mp4",
    fourcc,
    fps,
    (width, height)
)
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)
color = np.random.randint(
    0,
    255,
    (100, 3)
)

# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Error: Could not read first frame.")
else:

    # Convert first frame to grayscale
    old_gray = cv2.cvtColor(
        old_frame,
        cv2.COLOR_BGR2GRAY
    )

    # Detect good feature points
    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )
    mask = np.zeros_like(old_frame)

    frame_idx = 0
    plt.figure(figsize=(12, 6))
    while cap.isOpened():

        ret, frame = cap.read()
        if not ret:
            break

        frame_gray = cv2.cvtColor(
            frame,
```

```
            cv2.COLOR_BGR2GRAY
        )

        # If points disappear, detect new points
        if p0 is None or len(p0) < 10:

            p0 = cv2.goodFeaturesToTrack(
                old_gray,
                mask=None,
                **feature_params
            )

        # Calculate Lucas-Kanade optical flow
        if p0 is not None:

            p1, st, err = cv2.calcOpticalFlowPyrLK(
                old_gray,
                frame_gray,
                p0,
                None,
                **lk_params
            )
            if p1 is not None:

                # Select good new points
                good_new = p1[st == 1]
                # Select corresponding old points
                good_old = p0[st == 1]
                # Draw motion trajectories
                for i, (new, old) in enumerate(
                    zip(good_new, good_old)
                ):
                    # New position
                    a, b = new.ravel()
                    c, d = old.ravel()

                    mask = cv2.line(
                        mask,
                        (int(a), int(b)),
                        (int(c), int(d)),
                        color[i % 100].tolist(),
                        2
                    )

                    frame = cv2.circle(
                        frame,
                        (int(a), int(b)),
                        4,
                        color[i % 100].tolist(),
                        -1
                    )

                # Update points
                p0 = good_new.reshape(-1, 1, 2)

        # Combine original frame and trajectories
        img = cv2.add(frame, mask)

        # Write frame to output video
        out_lk.write(img)

        # Update previous frame
        old_gray = frame_gray.copy()

        # Display every 30th frame
        if frame_idx % 30 == 0:

            clear_output(wait=True)

            plt.imshow(
                cv2.cvtColor(
                    img,
                    cv2.COLOR_BGR2RGB
                )
            )

            plt.title(
                f"Lucas-Kanade Sparse Optical Flow "
                f"Frame: {frame_idx}/{total_frames}"
            )

            plt.axis("off")
            display(plt.gcf())

        frame_idx += 1
cap.release()
out_lk.release()
plt.close()

print(
    f"Lucas-Kanade processing finished."
```

```
)
print(
    f"Saved {frame_idx} frames to "
    f"'lucas_kanade_output.mp4'"
)
```




```
Lucas-Kanade processing finished.
Saved 125 frames to 'lucas_kanade_output.mp4'
```

In [ ]:
```
# Re-open the video         #Pragya singahl, CU24220050, 43
cap = cv2.VideoCapture(video_path)
out_dense = cv2.VideoWriter(
    "farneback_output.mp4",
    fourcc,
    fps,
    (width, height)
)
ret, frame1 = cap.read()

if not ret:

    print("Error: Could not read first frame.")

else:
    # Convert first frame to grayscale
    prvs = cv2.cvtColor(
        frame1,
        cv2.COLOR_BGR2GRAY
    )

    # Create HSV image
    hsv = np.zeros_like(frame1)
# Set saturation to maximum
hsv[..., 1] = 255
frame_idx = 0
plt.figure(figsize=(12, 6))
# Process video
while cap.isOpened():

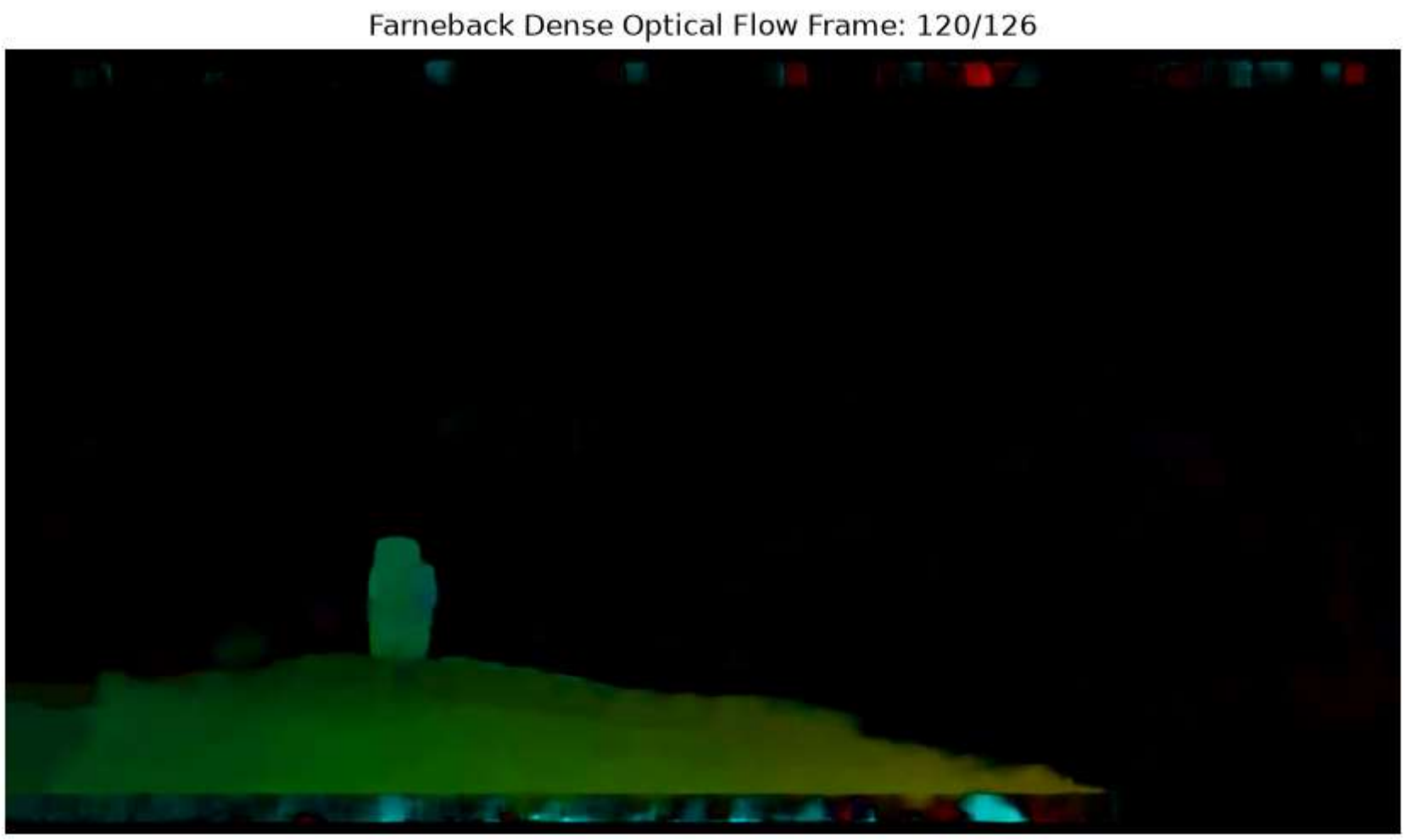
    ret, frame2 = cap.read()

    if not ret:
        break

    # Convert current frame to grayscale
    next_gray = cv2.cvtColor(
        frame2,
        cv2.COLOR_BGR2GRAY
    )
    # Calculate Farneback dense optical flow
    flow = cv2.calcOpticalFlowFarneback(
        prvs,
        next_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
```

```
        )
        # Convert flow vectors into magnitude and angle
        mag, ang = cv2.cartToPolar(
            flow[..., 0],
            flow[..., 1]
        )
        # Direction of motion
        hsv[..., 0] = (
            ang * 180 / np.pi / 2
        ).astype(np.uint8)
        # Magnitude of motion
        hsv[..., 2] = cv2.normalize(
            mag,
            None,
            0,
            255,
            cv2.NORM_MINMAX
        ).astype(np.uint8)
        # Convert HSV visualization to BGR
        bgr_flow = cv2.cvtColor(
            hsv,
            cv2.COLOR_HSV2BGR
        )
        # Write output frame
        out_dense.write(bgr_flow)
        # Update previous frame
        prvs = next_gray
        # Display every 30th frame
        if frame_idx % 30 == 0:
            clear_output(wait=True)
            plt.imshow(
                cv2.cvtColor(
                    bgr_flow,
                    cv2.COLOR_BGR2RGB
                )
            )
            plt.title(
                f"Farneback Dense Optical Flow "
                f"Frame: {frame_idx}/{total_frames}"
            )
            plt.axis("off")
            display(plt.gcf())

        frame_idx += 1
# Release resources
cap.release()
out_dense.release()
plt.close()
print(
    "Farneback Dense Optical Flow processing finished."
)
print(
    f"Saved {frame_idx} frames to "
    f"'farneback_output.mp4'"
)
```




```
Farneback Dense Optical Flow processing finished.
Saved 125 frames to 'farneback_output.mp4'
```